

1. Регистрация пользователя в банке

Цель:

Привязать биометрию пользователя к системе без хранения исходных данных лица.

Шаги:

1. Сканирование биометрии:

- Пользователь приходит в банк и сканирует лицо на терминале, оборудованном **TEE (Trusted Execution Environment)**.
- Пример оборудования: Специализированный киоск с камерой высокого разрешения и чипом **TrustZone**.

2. Генерация биометрического шаблона:

- Получение сырых данных лица (1000 ключевых точек).
- Нормализация данных (коррекция освещения, угла поворота).
- Применение алгоритма хеширования (например, SHA-3-512) с **уникальной солью**, генерируемой в TEE.

```
Биометрический_шаблон = SHA3(Соль_TEE + Данные_лица)
```

- **Соль_TEE** никогда не покидает TEE терминала

3. Создание секретного ключа банка:

- В TEE банковского терминала генерируется секретный ключ (**AES-256**)
- Пример: *Секретный_ключ_банка = 7f3a...d9b1 (256 бит)*.

4. Генерация refreshToken:

- refreshToken создается как HMAC от биометрического шаблона с использованием секретного ключа банка:

```
refreshToken = HMAC(Секретный_ключ_банка, Биометрический_шаблон)
```

- refreshToken шифруется с использованием алгоритма AES-256-GCM и сохраняется на сервере банка.

```
Зашифрованный_refreshToken = AES_GCM_Encrypt(refreshToken, Ключ_TEE)
```

- **Ключ_TEE** хранится только в TEE банковского терминала.

5. Уничтожение промежуточных данных:

- Исходные данные лица, биометрический шаблон и соль удаляются из памяти сразу после генерации refreshToken.

```
*Алексей сканирует лицо в банке. TEE терминала генерирует:  
• Соль_TEE = "a5e2...f8d3"  
• Биометрический_шаблон = SHA3("a5e2...f8d3" + данные_лица) → "BioHash_Alex".  
• Секретный_ключ_банка = "7f3a...d9b1".  
• refreshToken = HMAC("7f3a...d9b1", "BioHash_Alex") → "RT_123".  
Сервер сохраняет "RT_123" в зашифрованном виде: "Encrypted_RT_123"*
```

2. Оплата в магазине

Цель:

Подтвердить личность через биометрию, не передавая данные лица и не используя пароли.

Шаги:

1. Инициализация транзакции:

- Пользователь выбирает товар и нажимает "Оплатить биометрией".

2. Запрос nonce у сервера:

- Терминал магазина отправляет запрос на сервер банка.
- Сервер генерирует nonce (одноразовый код) с временной меткой:

```
nonce = timestamp + случайные_данные (128 бит)
```

Пример:

- timestamp = 1678901234 (UNIX-время).
- случайные_данные = "a1b2c3d4e5f6".
- nonce = "1678901234|a1b2c3d4e5f6".

3. Генерация динамического правила (DyRule):

- Сервер вычисляет DyRule с использованием секретного ключа банка:

```
DyRule = HMAC(Секретный_ключ_банка, nonce)
```

Пример:

- HMAC("7f3a...d9b1", "1678901234|a1b2c3d4e5f6") → "DyRule_AbC".

4. Подпись DyRule:

- Сервер подписывает DyRule своим приватным RSA-ключом для защиты от подмены:

```
Подпись_DyRule = RSA_Sign(Приватный_ключ_сервера, DyRule)
```

5. Передача данных терминалу:

- Сервер отправляет терминалу:
 - nonce.
 - DyRule.
 - Подпись_DyRule.

6. Проверка подписи терминалом:

- Терминал проверяет подпись с помощью публичного ключа сервера:

```
bool valid = RSA_Verify(Публичный_ключ_сервера, DyRule, Подпись_DyRule)
```

- Если подпись неверна, транзакция прерывается.

7. Сканирование биометрии:

- Пользователь подносит лицо к терминалу магазина.
- TEE терминала:
 - Собирает данные лица.
 - Генерирует биометрический шаблон (аналогично регистрации).
 - Пример: "BioHash_Alex".

8. Создание accessToken:

- ТЕЕ терминала вычисляет:

```
accessToken = HMAC(Биометрический_шаблон, DyRule)
```

Пример:

- $\text{HMAC}(\text{"BioHash_Alex"}, \text{"DyRule_AbC"}) \rightarrow \text{"Tok_123"}$.

9. Отправка данных на сервер:

- Терминал отправляет серверу:
 - `accessToken ("Tok_123")`.
 - `nonce ("1678901234|a1b2c3d4e5f6")`.

3. Верификация на сервере

Цель:

Убедиться, что операция легальна, и подтвердить списание средств.

Шаги:

1. Проверка времени жизни nonce:

- Сервер извлекает временную метку из nonce:

```
timestamp = 1678901234
```

- Вычисляет разницу между текущим временем и timestamp:

```
разница = текущее_время - timestamp
```

- Если разница > 2 минуты → транзакция отклоняется.

2. Проверка уникальности nonce:

- Сервер проверяет, не использовался ли nonce ранее.
- Если nonce есть в списке использованных → транзакция отклоняется.

3. Генерация DyRule на сервере:

- Сервер повторно вычисляет DyRule:

```
DyRule = HMAC(Секретный_ключ_банка, nonce)
```

Пример: "DyRule_AbC".

4. Расшифровка refreshToken:

- Сервер запрашивает ТЕЕ банка расшифровать refreshToken:

```
refreshToken = AES_GCM_Decrypt(Encrypted_RT_123, Ключ_ТЕЕ)
```

5. Генерация ожидаемого токена:

- Сервер вычисляет ожидаемый accessToken:

```
Ожидаемый_токен = HMAC(refreshToken, DyRule)
```

Пример: $\text{HMAC}(\text{"RT_123"}, \text{"DyRule_AbC"}) \rightarrow \text{"Tok_123"}$.

6. Сравнение токенов:

- Если `accessToken` == `Ожидаемый_токен` → оплата подтверждена.
- Иначе → транзакция отклоняется.

7. Обновление списка использованных nonce:

- Сервер сохраняет nonce в базу использованных.

```
*Сервер получает "Tok_123" и "1678901234|a1b2c3d4e5f6".
• Проверяет, что nonce актуален (1678901234 - текущее время ≤ 2 мин).
• Вычисляет DyRule = "DyRule_AbC".
• Расшифровывает refreshToken → "RT_123".
• Сравнивает HMAC("RT_123", "DyRule_AbC") с "Tok_123" → совпадение.
  → Средства списываются.*
```

1. Восстановление доступа через МФЦ (другое официальное учреждение)

Цель:

Обеспечить восстановление доступа при утере устройства или компрометации данных.

Шаги:

1. Аутентификация в МФЦ:

- Пользователь предъявляет паспорт и сканирует лицо.

2. Проверка личности:

- Сотрудник МФЦ сверяет фото в паспорте с лицом пользователя.

3. Инициализация сброса:

- Сотрудник через защищенный интерфейс отправляет запрос на сервер.

4. Удаление старого refreshToken:

- Сервер помечает старый refreshToken как недействительный.

5. Повторная регистрация:

- Пользователь заново сканирует лицо → генерируется новый биометрический шаблон.
- Создается новый секретный ключ банка и refreshToken.

1. Пример атаки и защита

Сценарий:

Злоумышленник перехватывает:

- `nonce` = "1678901234|a1b2c3d4e5f6"
- `DyRule` = "DyRule_AbC"
- `accessToken` = "Tok_123".

Попытка атаки:

1. Злоумышленник пытается повторно использовать nonce:
 - Сервер обнаруживает, что nonce уже использован → отклоняет запрос.
2. Злоумышленник пытается создать новый accessToken:
 - Без биометрического шаблона ("BioHash_Alex") и секретного ключа банка ("7f3a...d9b1") сгенерировать валидный токен невозможно.
3. Злоумышленник пытается подделать DyRule:
 - Без секретного ключа банка невозможно создать правильное DyRule для нового nonce.

Защитные механизмы

Компонент	Защита
Биометрический шаблон	Необратимый хеш с солью, уничтожается сразу после использования.
Секретный ключ банка	Хранится в TEE, недоступен для чтения/записи извне.
RefreshToken	Шифруется AES-256-GCM с ключом из TEE.
Nonce	Одноразовый, с временной меткой, проверяется на сервере.
Динамическое правило	Зависит от секретного ключа банка, подписывается сервером.
AccessToken	Генерируется только при наличии биометрии и актуального DyRule.